

DSA Explorer and Visualiser App

Data Structure and Algorithms Visualisation Using PyGame

Submitted by: Nidup Dema

Date: 16 May 2026

Introduction

The purpose of this project is to develop an interactive Data Structures and Algorithms(DSA) Explorer and Visualiser application using Python and Pygame. The application allows users to learn and understand important data structures and algorithms through visual animations and interactive controls.

The project developed using a modular design approach, where each module focuses on specific DSA concept. The final application includes three fully implemented and tested module:

- Data structures Playground
- Sorting Visualiser
- Graph Traversal Visualiser

The application also includes automated testing using Python's unittest module to ensure correctness and reliability.

Project Overview

The DSA Explorer and Visualiser App combines multiple data structure and algorithm visualisations into one interactive application. The project uses PyGame for graphical rendering, animations, event handling, and user interaction.

The application provides users with the ability to:

- Explore data structures visually
- Interact with algorithms using buttons and controls
- Observe algorithm behaviour step-by-step
- Understand traversal and sorting processes
- Learn DSA concepts in an engaging and interactive way

The project is organised into independent modules for maintainability and scalability. Each module contains its own logic and PyGame interface while sharing a common design structure.

The application currently includes:

1. Data Structures Playground
2. Sorting Visualiser
3. Graph Traversal Visualiser

App Design and Layout

The application begins with a main menu screen that allows the user to navigate between different modules. Each module opens in its own PyGame screen and contains dedicated interactive controls.

The layout design focuses on simplicity, readability, and usability. A consistent colour scheme and font style are used throughout the project to maintain a clean user interface.

The main menu contains:

- Data Structures button
- Sorting button
- Graphs button
- Quit button

Each module contains:

- A title
- Instructions
- Interactive buttons
- Visual animation area
- Back button to return to the main menu

PyGame event handling is used to detect mouse clicks and manage transitions between modules.

Figure 1: Main Menu Interface

DSA Explorer and Visualiser App

Choose a module to explore

Data Structures

Sorting

Graphs

Quit

The modular structure improves code readability and allows future modules to be added easily without affecting existing components.

Module 1 – Data Structures Playground

Overview

The Data Structures Playground demonstrates the behaviour of two important linear data structures:

- Stack
- Queue

The purpose of this module is to visually demonstrate insertion and removal operations while showing the ordering principles used by each data structure.

Stack Visualisation

A stack follows the LIFO (Last In First Out) principle. This means the most recently inserted item is removed first.

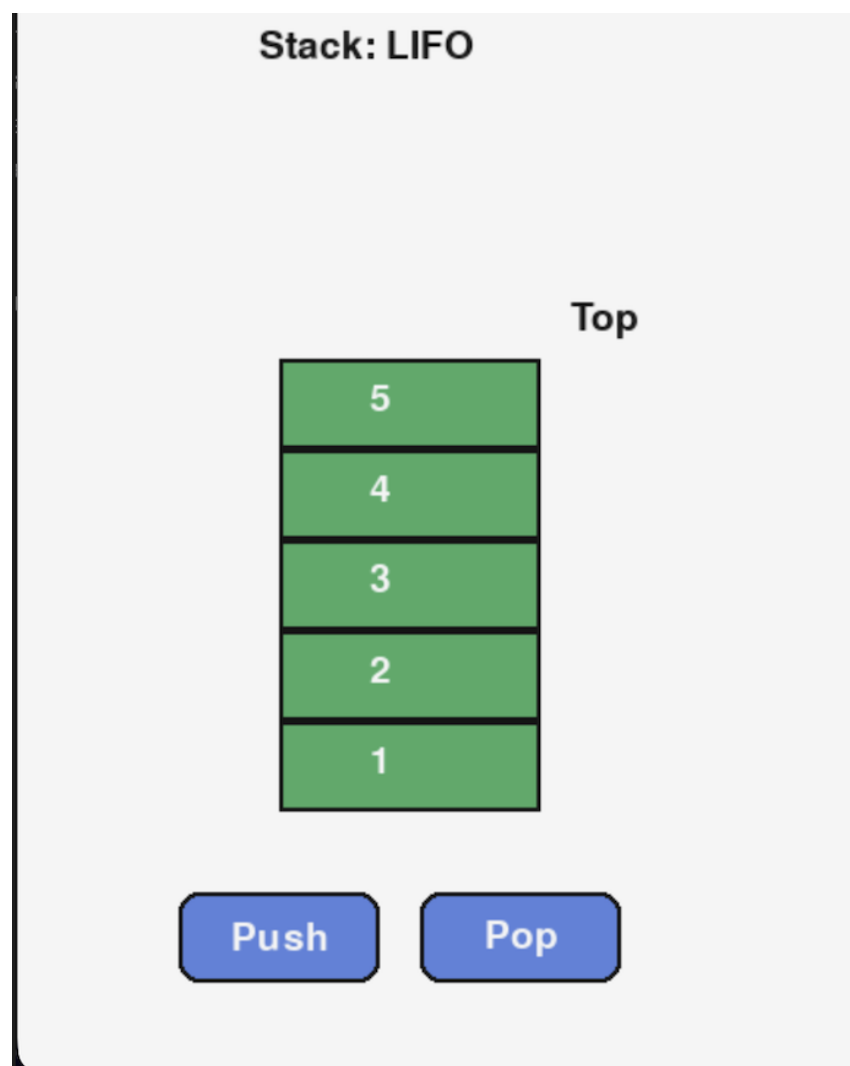
The stack module includes:

- Push operation
- Pop operation
- Visual stack growth
- Top indicator

When the user presses the Push button, a new item is added to the top of the stack. When the Pop button is pressed, the most recent item is removed.

The stack is displayed vertically so users can clearly observe the LIFO behaviour.

Figure 2: Stack Visualisation



Queue Visualisation

A queue follows the FIFO (First In First Out) principle. This means the earliest inserted item is removed first.

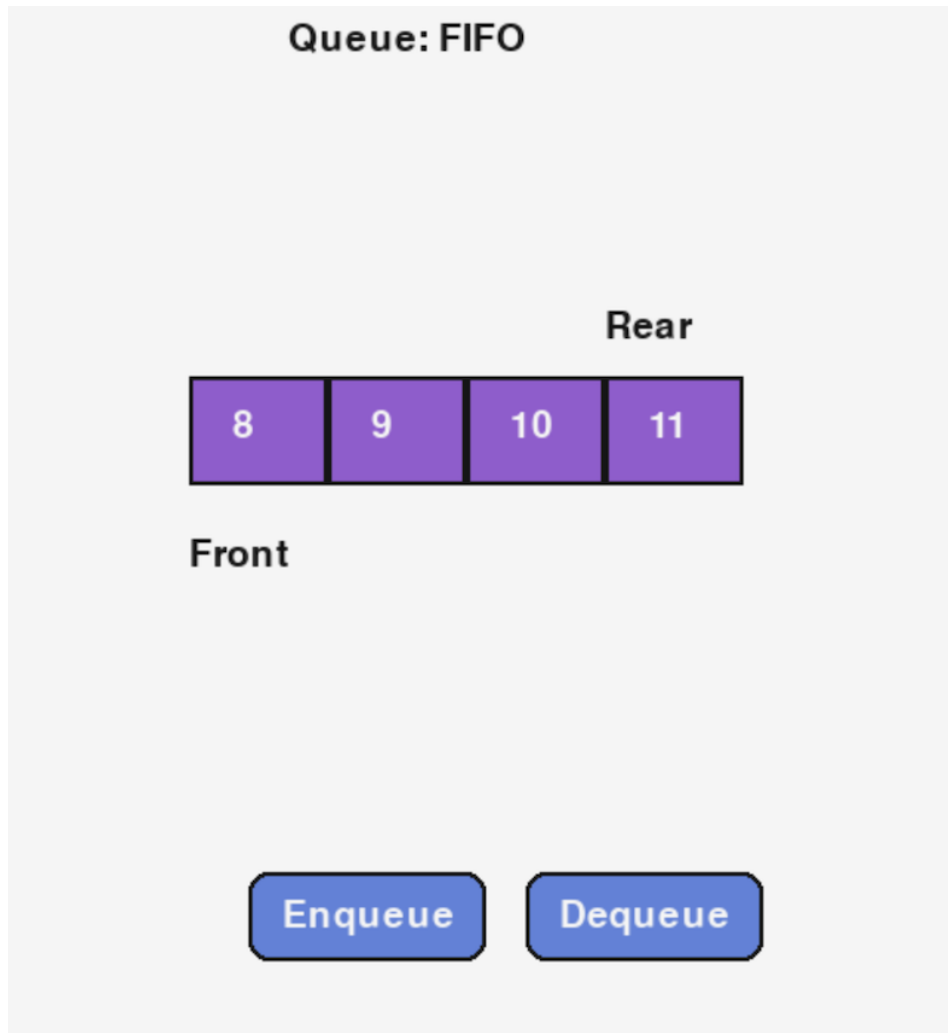
The queue module includes:

- Enqueue operation
- Dequeue operation
- Front and rear indicators

When the user presses the Enqueue button, an item is added to the rear of the queue. When the Dequeue button is pressed, the front item is removed.

The queue is displayed horizontally to clearly demonstrate FIFO ordering.

Figure 3: Queue Visualisation



Concepts Demonstrated

This module demonstrates:

- LIFO ordering
- FIFO ordering
- Stack operations
- Queue operations
- Interactive user input

The module successfully visualises how different linear data structures manage data internally.

Module2 – Sorting Visualiser

Overview

The Sorting Visualiser demonstrates two common sorting algorithms:

- Bubble Sort
- Selection Sort

Sorting algorithms are important in computer science because they organise data efficiently and are used in many real-world applications.

The module uses animated bars to represent array elements. The bars change position and colour during sorting to visually demonstrate comparisons and swaps.

Bubble Sort

Bubble Sort repeatedly compares adjacent elements and swaps them if they are in the wrong order.

The process continues until the array becomes fully sorted.

The Bubble Sort visualiser includes:

- Animated comparisons
- Swapping visualisation
- Step-by-step sorting progression

Orange bars indicate elements currently being compared.

Figure 4: Bubble Sort Animation



Bubble Sort is easy to understand but inefficient for large datasets due to its high time complexity.

Selection Sort

Selection Sort repeatedly searches for the smallest remaining element and places it into its correct sorted position.

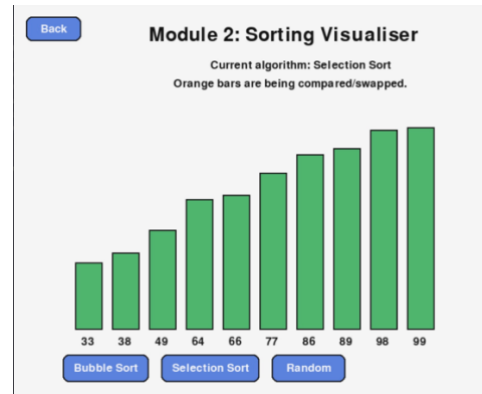
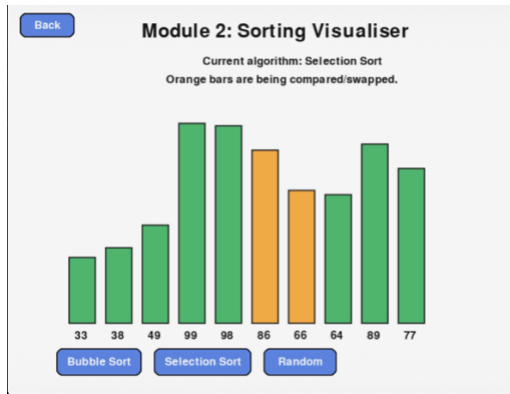
The algorithm divides the array into:

- Sorted portion
- Unsorted portion

The Selection Sort visualiser demonstrates:

- Minimum element selection
- Element swapping
- Progressive array sorting

Figure 5: Selection Sort Animation



Selection Sort performs fewer swaps than Bubble Sort but still has quadratic time complexity.

Concepts Demonstrated

This module demonstrates:

- Array manipulation
- Comparison-based sorting
- Swapping operations
- Sorting visualisation
- Step-by-step algorithm execution

Graph Traversal Visualiser

Overview

The Graph Traversal Visualiser demonstrates graph traversal algorithms using an interactive graph.

The user can select a start node and run:

- Breadth First Search (BFS)
- Depth First Search (DFS)

Nodes are displayed as circles connected by edges. During traversal, visited nodes change colour to visualise the algorithm progression.

Breadth First Search (BFS)

Breadth First Search explores nodes level-by-level using a queue data structure.

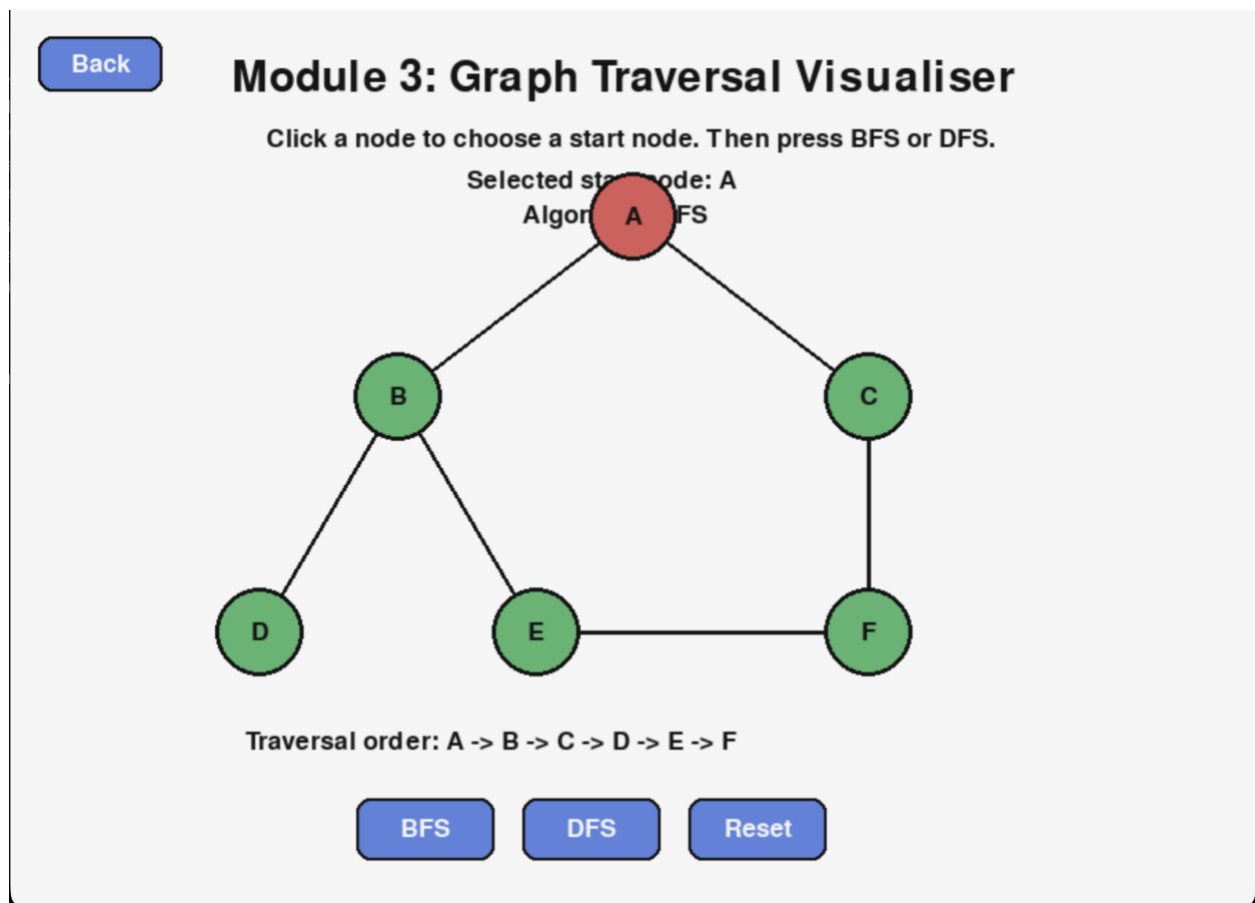
The BFS algorithm:

1. Starts from a selected node
2. Visits all neighbouring nodes
3. Expands outward level-by-level

The BFS visualisation demonstrates:

- Queue-based traversal
- Sequential exploration
- Visited node highlighting

Figure 6: BFS Traversal



BFS is commonly used in:

- Shortest path problems
- Network traversal
- Pathfinding systems

Depth First Search (DFS)

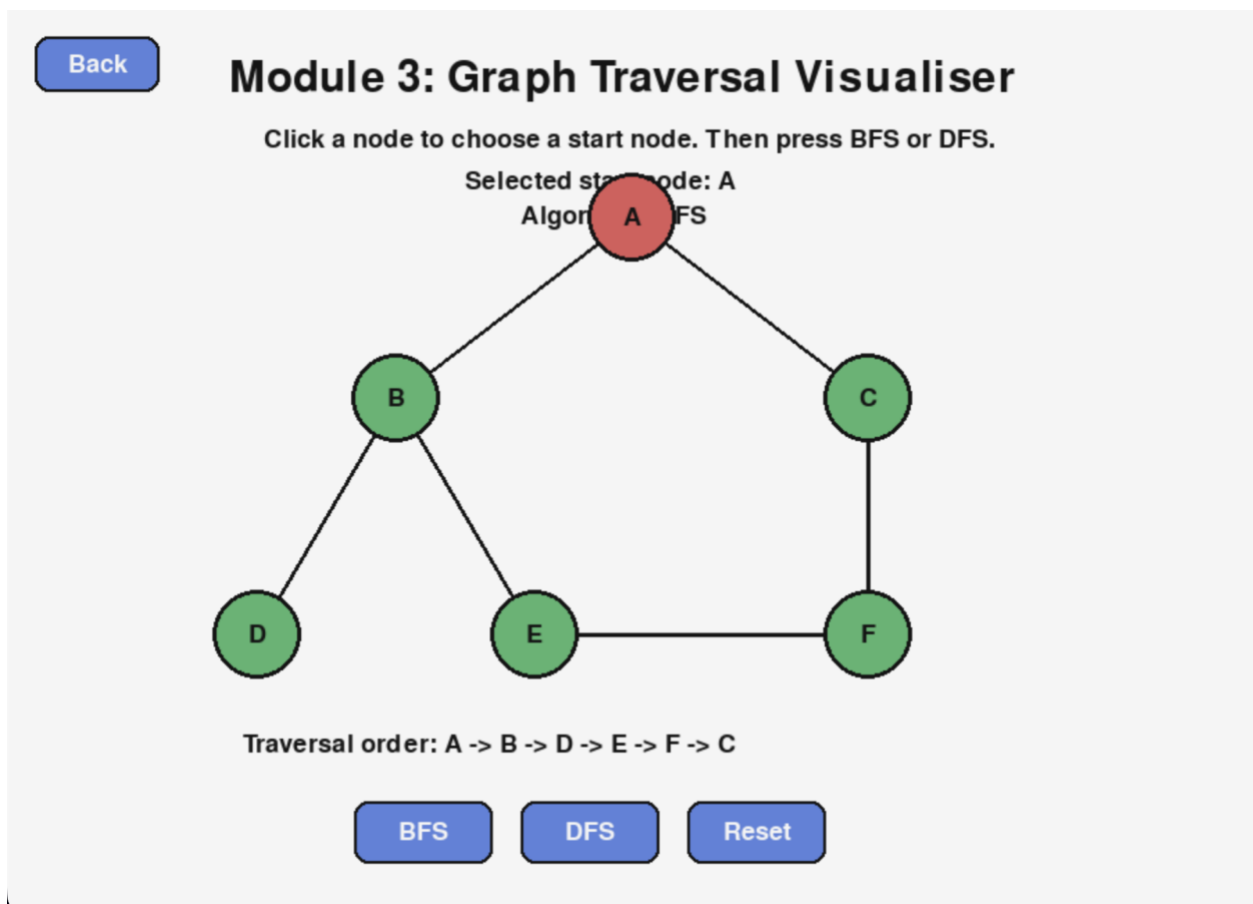
Depth First Search explores one path as deeply as possible before backtracking.

DFS can be implemented using recursion or a stack.

The DFS visualisation demonstrates:

- Depth-first exploration
- Traversal backtracking
- Node visitation order

Figure 7: DFS Traversal



DFS is commonly used in:

- Graph analysis
- Maze solving
- Topological sorting

Concepts Demonstrated

This module demonstrates:

- Graph structures
- Node traversal
- BFS algorithm
- DFS algorithm
- Queue and stack behaviour in graph traversal

7. Automated Testing

Automated testing was implemented using Python's unittest module to validate the correctness of algorithms and data structures.

The tests verify:

- Stack push and pop operations
- Queue enqueue and dequeue operations
- Bubble Sort correctness
- Selection Sort correctness
- BFS traversal order
- DFS traversal order

The automated testing system improves software reliability and ensures algorithms behave as expected.

The following command was used to execute tests:

```
DSA_Explorer_Project % python3 -m unittest discover tests
```

The project successfully passed all automated tests.

Figure 8: Automated Test Results

The test output confirmed:

```
.....  
-----  
Ran 9 tests in 0.000s  
  
OK
```

Conclusion

The DSA Explorer and Visualiser App successfully demonstrates important data structures and algorithms using interactive PyGame visualisation.

The project achieved the following objectives:

- Implemented three fully functional DSA modules
- Provided interactive user controls
- Demonstrated algorithm animations
- Applied modular software design
- Implemented automated testing
- Created a scalable project structure

The project improved understanding of:

- Data structures
- Sorting algorithms
- Graph traversal algorithms
- PyGame programming
- Software testing

Future improvements could include:

- Linked Lists
- Binary Search Trees
- Heap Visualisers
- Pathfinding Algorithms
- Dynamic Programming Challenges

Overall, the project successfully combines education, interaction, and visualisation into a single modular application.

Github Repository Link

<https://github.com/nidupdema15-png/DSA-Explorer-Visualizer.git>

